

Análise Exploratória de Dados

Capítulo 5 — Da fonte ao visual

Cayo Oliveira

Sumário

5	Da fonte ao visual	3
5.1	A pergunta vem antes da fonte	3
5.2	Sistemas fonte e ingestão preservável	5
5.3	SOR, SOT e SPEC: responsabilidades separadas	6
5.4	Armazenamento é consequência do acesso	8
5.5	Grão, chaves e cardinalidade antes do join	9
5.6	Modelagem dimensional para explorar sem duplicar	10
5.7	SQL e pandas depois do desenho	11
5.8	Qualidade distribuída por camada	13
5.9	Catálogo, linhagem e camada semântica	14
5.10	Do modelo ao dashboard com acesso seguro	16
5.11	Fechamento: da divergência à contestação	17

Capítulo 5

Da fonte ao visual: arquitetura de dados para AED

Uma rede varejista vende na loja física, no aplicativo próprio e em um marketplace. Na reunião de segunda-feira, três painéis respondem à pergunta “qual foi a receita de agosto?” com números diferentes. O sistema do caixa soma tudo o que passou no ponto de venda; o aplicativo desconta cupons, mas ainda não recebeu todas as devoluções; o marketplace informa somente o valor repassado depois de comissão. Os três cálculos podem estar tecnicamente corretos e, ao mesmo tempo, representar conceitos incompatíveis.

Nos capítulos anteriores, aprendemos a formular a pergunta, proteger o grão, calcular centros e dispersões, observar distribuição, ausências e extremos. Agora veremos o que precisa existir entre a origem e o gráfico para que essas análises continuem significando a mesma coisa. Arquitetura de dados para AED não começa pela escolha de nuvem ou ferramenta: começa pela decisão que precisa ser tomada e pelo contrato da medida que sustentará essa decisão.

O percurso será pergunta $\rightarrow$ fonte $\rightarrow$ ingestão $\rightarrow$ SOR $\rightarrow$ SOT $\rightarrow$ SPEC $\rightarrow$ análise/-dashboard. SOR preservará o que os sistemas registraram; SOT produzirá entidades e significados conformados; SPEC entregará um conjunto especializado para uma pergunta ou produto analítico. Em cada transição, registraremos grão, chaves, regras, qualidade, responsável e linhagem.

O encontro foi desenhado para três horas. Na primeira hora, reconstruiremos fontes, ingestão e camadas; na segunda, modelaremos fatos, dimensões, chaves e junções; na terceira, conectaremos camada semântica, qualidade, catálogo, linhagem, segurança e consumo. SQL e pandas aparecerão depois da definição conceitual para verificar o desenho, não para decidir silenciosamente qual conceito de receita venceu.

Ao final, o leitor deve conseguir desenhar uma arquitetura rastreável da pergunta à decisão e explicar onde cada controle atua. O resultado não será uma lista de tecnologias favoritas, mas uma sequência em que qualquer número do dashboard pode ser ligado à fonte, à transformação, à definição e ao responsável. Essa rastreabilidade será indispensável quando, na Unidade II, uma IA responder perguntas diretamente sobre os dados.

5.1 A pergunta vem antes da fonte

A frase “qual foi a receita de agosto?” ainda não é uma especificação analítica. Receita pode significar valor bruto do pedido, valor depois de desconto, valor faturado, valor entregue, valor líquido de devoluções, valor recebido ou repasse depois de taxas. Agosto

pode ser definido pela data do pedido, do pagamento, da entrega, do estorno ou da competência contábil. Antes de procurar tabela, registramos qual decisão será tomada e qual evento deve entrar no cálculo.

Suponha que a diretoria queira dimensionar demanda comercial. Nesse caso, pedidos confirmados pelo valor antes de devoluções podem ser úteis. Se finanças pretende estimar caixa, pagamento liquidado e estorno importam mais. Se operações avalia capacidade, itens entregues e região de destino são centrais. Um dashboard não resolve essa divergência com um filtro; cada intenção exige uma medida nomeada e um denominador coerente.

O contrato inicial pode dizer: “receita líquida entregue é a soma do valor dos itens entregues no período, menos descontos do item e devoluções aprovadas, na moeda BRL, atribuída ao mês da entrega”. A frase define evento, componentes, estado, moeda e tempo. Também declara o que fica de fora: frete, pedido cancelado, devolução apenas solicitada e venda ainda não entregue. Esse texto se tornará teste e metadado.

A unidade de análise é pedido entregue por mês quando olhamos o total mensal, mas a unidade de observação necessária pode ser item do pedido. Devolução ocorre por item, desconto pode variar por linha e um pedido pode conter produtos de categorias distintas. Se agregarmos cedo demais no nível de pedido, perdemos a possibilidade de atribuir devolução ao produto correto. Se mantivermos item até o consumo sem governar a agregação, podemos duplicar pagamentos.

Dimensões de corte também pertencem ao contrato. “Por canal” exige uma classificação estável quando o cliente compra no aplicativo e retira na loja: o canal será origem do pedido, local da entrega ou responsável pela receita? “Por região” pode significar loja, endereço do cliente ou destino. O campo existe na fonte, mas a pergunta decide qual papel ele exerce. Quando mais de um papel é útil, publicamos dimensões qualificadas em vez de uma coluna genérica.

Critérios de aceitação tornam o contrato testável. Podemos exigir que todo pedido entregue possua ao menos um item entregue, que a receita líquida nunca inclua cancelados, que a soma por canal reconcilie com o total e que a atualização informe o maior evento processado. Uma frase de negócio sem testes fica dependente da memória; um teste sem definição apenas congela comportamento, mesmo que esteja conceitualmente errado.

Também declaramos o corte de atualização. Um painel atualizado às 8h com dados até 7h45 não deve ser comparado a outro com fechamento do dia anterior sem aviso. Latência, completude e janela de reproprocessamento fazem parte do significado. Em uma fonte sujeita a devoluções tardias, agosto pode continuar mudando em setembro; a versão do fechamento precisa ser conhecida.

A pergunta funciona como teste de relevância para cada coluna. Nome do operador de caixa talvez seja necessário para auditoria, mas não para o dashboard executivo. Identificador de pedido, item, canal, data, estado e valores são essenciais para rastrear e calcular. Remover uma coluna não é apenas reduzir arquivo: é decidir quais investigações futuras ainda serão possíveis, por isso a camada bruta deve conservar a fonte antes das escolhas analíticas.

Explicação guiada

Aluno, preste atenção aqui: “a fonte correta” depende da medida. O caixa pode ser autoridade sobre venda registrada; o adquirente, sobre pagamento liquidado; a logística, sobre entrega. Fonte de verdade não significa uma tabela universal para todas as perguntas.

Tabela 5.1: Uma palavra de negócio pode esconder medidas diferentes.

Medida	Regra resumida	Decisão apoiada
Receita bruta pedida	itens confirmados antes de descontos e devoluções	demanda comercial
Receita líquida entregue	itens entregues menos descontos e devoluções aprovadas	desempenho do canal
Caixa recebido	pagamentos liquidados menos estornos	liquidez financeira
Repasse marketplace	valor transferido depois de comissão	conciliação do parceiro

Síntese da trilha

Então, aluno, veja só o que você aprendeu aqui: antes de conectar dados, transforme a pergunta em contrato. Evento, grão, tempo, estado, componentes e exclusões determinam quais fontes e controles serão necessários.

5.2 Sistemas fonte e ingestão preservável

Sistemas transacionais, também chamados OLTP, são otimizados para registrar operações curtas e concorrentes: criar pedido, autorizar pagamento, baixar estoque e atualizar entrega. Eles costumam normalizar dados para reduzir inconsistência e manter integridade. A estrutura que protege a transação nem sempre é a melhor para explorar milhões de linhas, comparar longos períodos ou calcular medidas em várias dimensões.

Sistemas analíticos, ou OLAP, são organizados para leitura, agregação e comparação. Em vez de atualizar uma linha de pedido a cada evento, podem manter fatos históricos e dimensões que facilitam cortes por tempo, canal, produto e região. OLTP e OLAP não são rivais: um registra a operação; o outro reconstrói evidência para análise sem sobrecarregar o caminho da venda.

No caso varejista, as fontes incluem banco do ponto de venda, API do aplicativo, arquivo diário do marketplace, gateway de pagamento, sistema de logística e cadastro mestre de produtos. Cada fonte possui dono, frequência, fuso, chave e limite. O CSV do marketplace pode chegar novamente com correções; a API pode paginar; o banco pode registrar alterações; o pagamento pode ter múltiplas tentativas para o mesmo pedido.

Ingestão é o movimento controlado da fonte para a plataforma analítica. Em lote, dados são capturados em intervalos, como arquivo diário ou extração horária. Em fluxo, eventos chegam continuamente. O critério não é modernidade: um fechamento financeiro diário pode preferir lote reproduzível; monitoramento de fraude pode exigir baixa latência. Frequência deve corresponder ao tempo da decisão e à capacidade de corrigir eventos tardios.

Uma ingestão confiável registra metadados operacionais além dos campos de negócio: `source_system`, `ingested_at`, `batch_id`, nome e hash do arquivo, versão do esquema e posição de origem. Esses campos permitem responder quando o registro chegou, de onde veio e se um lote foi repetido. Sem eles, uma duplicação causada por reenvio parece venda adicional.

Idempotência significa que repetir a mesma ingestão não produz efeito acumulado indevido. Se o arquivo de 03/09 for processado duas vezes, a camada posterior deve

reconhecer lote e chave, substituindo ou ignorando conforme regra. Em eventos mutáveis, podemos usar chave e versão; em arquivos imutáveis, hash e caminho. A estratégia precisa ser testada antes da falha, não improvisada depois que a receita dobra.

Mudanças de esquema também fazem parte da ingestão. O marketplace pode renomear `amount` para `gross_amount`, mudar decimal para texto ou adicionar moeda. Aceitar tudo silenciosamente quebra métricas; rejeitar qualquer coluna nova pode interromper um fluxo saudável. Contratos classificam alterações compatíveis e incompatíveis, geram alerta e preservam o lote para reprocessamento.

Extração não deve apagar a origem. Converter data, padronizar texto e imputar ausências já são transformações. A antiga prática de abrir um CSV, limpar e sobrescrever `arquivo_limpo.csv` atende uma demonstração curta, mas perde rastreabilidade. Em arquitetura, guardamos a captura original e produzimos novas camadas por transformação versionada.

Tabela 5.2: Fontes do caso e controles mínimos de ingestão.

Fonte	Ritmo	Chave de origem	Controle
Ponto de venda	eventos	loja + cupom + item	posição e duplicidade
Aplicativo	API horária	order id + line id	paginação e versão
Marketplace	arquivo diário	partner order + item	hash, lote e reenvio
Pagamentos	eventos	transaction id	tentativa, estado e estorno

Síntese da trilha

Então, aluno, veja só o que você aprendeu aqui: ingestão é um contrato de captura. Ritmo, idempotência, esquema, versão e metadados operacionais protegem a passagem da operação para a análise sem reescrever a história.

5.3 SOR, SOT e SPEC: responsabilidades separadas

Os nomes das camadas variam entre organizações. Neste curso, usaremos SOR como *System of Record* analítico preservado, SOT como *Source of Truth* conformada e SPEC como camada especializada de consumo. O valor pedagógico não está na sigla, mas na separação de responsabilidades: captura fiel, significado compartilhado e produto adequado à pergunta.

SOR recebe os dados próximos da forma de origem, com metadados de ingestão e histórico. Erros plausíveis, códigos antigos e valores ausentes permanecem visíveis. Podemos converter embalagem técnica para leitura segura, como armazenar JSON, mas não substituímos silenciosamente o valor de negócio. SOR responde “o que a fonte informou e quando recebemos?”.

SOT integra e conforma entidades. Identificadores de produto de canais diferentes são mapeados para uma chave corporativa; datas usam fuso acordado; estados ganham vocabulário comum; duplicatas são tratadas por regra; moeda é normalizada mantendo valor e taxa originais. SOT responde “qual representação governada usamos para comparar a empresa?”.

SPEC é uma especificação materializada para um consumidor ou finalidade: fato de receita líquida diária, base de análise de entregas, conjunto de características de um modelo ou visão para dashboard. Ela seleciona grão, dimensões, medidas e filtros autorizados.

SPEC não deve inventar significado local sem registro; deriva da SOT e aponta para a definição semântica usada.

Uma arquitetura pode chamar essas zonas de bronze, silver e gold, raw, curated e serving, ou ainda landing, integration e presentation. Os termos não são perfeitamente equivalentes em todas as empresas. Para evitar tradução automática, documentamos finalidade, mutabilidade, contrato e consumidores de cada camada. O desenho importa mais que a cor escolhida.

No caso de vendas, a SOR mantém `pos_receipts`, `app_orders`, arquivos do parceiro e eventos de pagamento separados. A SOT cria pedido, item, produto, cliente, canal, entrega, pagamento e devolução com chaves conformadas. A SPEC de receita entregue agrega ou mantém itens segundo o contrato da medida e expõe somente campos necessários ao painel.

Reprocessamento começa na camada mais antiga confiável. Se descobrimos que um código do marketplace foi mapeado ao produto errado, corrigimos a regra SOT e regeneramos SPEC. Não editamos a célula final do dashboard. A possibilidade de reconstrução diferencia uma arquitetura rastreável de uma sequência de arquivos manuais cujo estado depende da ordem em que alguém clicou.

A Figura 5.1 mostra o fluxo em duas linhas. A seta de volta simboliza contestação: se o dashboard divergir, percorremos SPEC, SOT, SOR e fonte até localizar definição, transformação ou atraso. Qualidade e linhagem atravessam todas as etapas; não são uma limpeza isolada no fim.

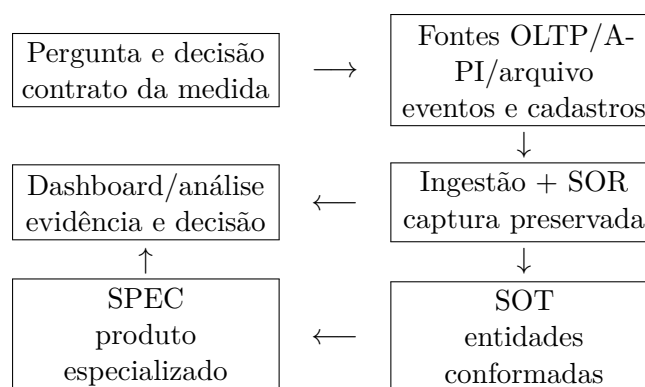


Figura 5.1: Arquitetura de AED: da pergunta à fonte e de volta à decisão contestável.

Explicação guiada

Aluno, preste atenção aqui: SOT não é “a única tabela verdadeira”. É a representação conformada e governada para um domínio. Autoridades operacionais continuam distribuídas, e SPEC adapta a verdade governada ao grão de uma pergunta sem perder linhagem.

Síntese da trilha

Então, aluno, veja só o que você aprendeu aqui: SOR preserva, SOT conforma e SPEC serve. Separar essas responsabilidades permite corrigir regras, reconstruir produtos e explicar por que o número chegou ao painel.

5.4 Armazenamento é consequência do acesso

Um data warehouse organiza dados integrados e históricos para consulta analítica, frequentemente com esquema definido antes do consumo e forte governança. Um data lake armazena arquivos e objetos em formatos variados, oferecendo flexibilidade e escala. Um lakehouse combina armazenamento aberto de objetos com recursos de tabela, transação, catálogo e desempenho analítico. Esses conceitos descrevem capacidades e padrões; nomes comerciais podem misturá-los.

Escolher entre eles não substitui modelagem. Um lake pode virar coleção opaca de arquivos sem catálogo; um warehouse pode conter dezenas de tabelas contraditórias; um lakehouse pode reproduzir ambos os problemas com tecnologia mais nova. Pergunta, contrato, camadas, qualidade e responsabilidade continuam necessários independentemente do mecanismo de armazenamento.

Formatos colunares, como Parquet, favorecem consultas que leem poucas colunas em muitas linhas e comprimem valores semelhantes. Bancos colunares e índices analíticos cumprem função relacionada. Para AED, isso permite explorar período e dimensões sem carregar textos não usados. CSV é interoperável e legível, mas não preserva tipos com a mesma robustez e costuma custar mais para grandes volumes.

Particionamento divide fisicamente dados por campo, como data de evento, para evitar ler todo o histórico. Particionar por uma categoria de altíssima cardinalidade, como pedido, cria arquivos pequenos e piora operação. A escolha precisa refletir filtros frequentes, volume e manutenção. Partição não corrige consulta sem predicado nem substitui organização lógica.

Snapshots e dados mutáveis exigem estratégia. Uma dimensão de produto pode mudar categoria; um pedido pode receber novo estado; uma devolução pode ser aprovada dias depois. Podemos conservar versões e validade temporal, ou reconstruir snapshots. Se sobrescrevermos o passado, o painel de agosto poderá mudar sem que a venda mude, apenas porque o cadastro atual foi aplicado retroativamente.

Histórico lentamente mutável é uma decisão semântica. Para analisar vendas conforme a categoria vigente na data, mantemos versões de dimensão com `valid_from` e `valid_to`. Para sempre usar a classificação atual, um mapeamento corrente basta. As duas perguntas são legítimas; misturá-las no mesmo indicador sem nome produz divergência.

Performance deve ser medida no caminho da decisão. Uma SPEC diária pode pré-agregar itens para painel executivo, enquanto a SOT preserva item para investigação. Materializar tudo em todas as combinações aumenta custo e risco de inconsistência. Materializar nada transfere cálculos repetidos para cada ferramenta. O equilíbrio considera frequência, latência, custo, reuso e possibilidade de reconstrução.

Retenção e acesso também moldam armazenamento. Dados pessoais não precisam permanecer indefinidamente em todas as camadas. SOR pode ter proteção mais restrita; SOT pode pseudonimizar identificadores; SPEC deve expor o mínimo necessário. Segurança não é filtro colocado apenas no dashboard, pois cópias exportadas podem contornar esse controle.

Síntese da trilha

Então, aluno, veja só o que você aprendeu aqui: warehouse, lake e lakehouse oferecem capacidades diferentes, mas nenhum produz verdade por si só. O desenho nasce do acesso, histórico, governança, custo e reconstrução necessários.

Tabela 5.3: Capacidades típicas e perguntas de escolha.

Padrão	Força típica	Pergunta crítica
Warehouse	integração e consulta governada	esquema atende exploração e histórico?
Lake	variedade e armazenamento econômico	há catálogo, contrato e controle de arquivos?
Lakehouse	tabelas sobre objetos e abertura	capacidades prometidas estão operacionais?
SPEC materializada	desempenho e simplicidade no consumo	regra é reconstruível e reutilizável?

5.5 Grão, chaves e cardinalidade antes do join

Grão é o que uma linha representa. Na tabela de itens, uma linha pode ser um item de pedido; em pagamentos, uma tentativa de transação; em devoluções, um evento por item; em metas, um canal por mês. Escrever essa frase no catálogo é a defesa mais simples contra somas duplicadas. Se não conseguimos completar “uma linha representa...”, ainda não estamos prontos para juntar.

Uma chave primária identifica unicamente a linha dentro de sua entidade. `order_id` identifica pedido, mas não item; para item, podemos precisar de (`order_id`, `line_id`). Chave natural vem do processo; chave substituta é criada pela plataforma para integrar versões ou fontes. Nenhuma delas elimina a necessidade de validar unicidade e ausência.

Chave estrangeira liga uma entidade a outra. `product_key` em fato de item aponta para dimensão produto. Integridade referencial significa que a referência encontra um membro válido, ou um membro técnico explicitamente tratado como desconhecido. Um join com 8% de produtos sem correspondência não deve seguir silenciosamente para gráfico por categoria.

Cardinalidade descreve quantas linhas de um lado correspondem ao outro: 1:1, 1:N ou N:N. Pedido para item é 1:N; pedido para tentativa de pagamento também pode ser 1:N. Juntar diretamente itens e pagamentos por pedido cria N:N e multiplica linhas. O SQL executa corretamente a instrução, mas a soma passa a contar combinações, não eventos econômicos.

Considere o pedido O101 com dois itens líquidos de R\$ 60 e R\$ 40 e dois eventos de pagamento, uma tentativa recusada e uma liquidação de R\$ 100. Um join bruto entre itens e pagamentos gera quatro linhas. Somar `item_value` retorna R\$ 200; somar eventos pode retornar R\$ 200 ou misturar tentativa recusada. A receita real continua R\$ 100.

A solução começa pela pergunta e pelo grão alvo. Para receita por pedido, agregamos itens válidos por pedido e pagamentos liquidados por pedido separadamente; depois juntamos duas linhas 1:1. Para análise por item, não distribuimos pagamento sem regra; mantemos valor econômico do item e usamos pagamento apenas como estado do pedido. Pré-agregação não é perda quando o produto especializado declara seu grão.

Tipos de join determinam quais chaves permanecem. ‘INNER JOIN’ mantém correspondências; ‘LEFT JOIN’ preserva todas as linhas da esquerda; ‘FULL JOIN’ permite auditoria de chaves exclusivas de ambos. A escolha não é estética. Para contar vendas mesmo sem cadastro de produto, um left join com categoria desconhecida preserva fatos e expõe falha; inner join reduziria receita silenciosamente.

Antes e depois do join, registramos contagem de linhas, número de chaves, soma

de medidas aditivas, taxa de correspondência e distribuição de multiplicidade. Se itens têm 10 mil linhas e o join retorna 12 mil sem regra prevista, houve fanout. Testes de reconciliação devem falhar antes de o dashboard atualizar.

Tabela 5.4: Diagnóstico manual do fanout no pedido O101.

Tabela	Grão	Linhas de O101	Soma aparente
Itens	item do pe- dido	2	R\$ 100
Pagamentos	tentativa	2	uma liquidação de R\$ 100
Join bruto	item × tenta- tiva	4	itens somam R\$ 200
Pré-agregado	pedido	1	receita R\$ 100

Explicação guiada

Aluno, preste atenção aqui: um join que executa não é necessariamente um join correto. Declare o grão dos dois lados, a cardinalidade esperada e os invariantes de contagem e soma antes de combinar tabelas.

Síntese da trilha

Então, aluno, veja só o que você aprendeu aqui: chaves conectam, mas cardinalidade decide se a conexão preserva medidas. Pré-agregação, membros desconhecidos e testes de reconciliação protegem o significado do fato.

5.6 Modelagem dimensional para explorar sem duplicar

Modelagem dimensional organiza eventos mensuráveis em fatos e contextos descritivos em dimensões. Uma fato de venda no grão item entregue pode conter quantidade, valor bruto, desconto e valor devolvido. Dimensões descrevem data, produto, cliente, canal e local. O esquema estrela facilita agregação e torna explícito por quais eixos a análise pode ser cortada.

Medidas aditivas podem ser somadas em todas as dimensões, como valor do item, desde que cada evento apareça uma vez. Medidas semi-aditivas, como saldo de estoque, podem ser somadas por produto, mas não ao longo do tempo sem regra. Razões, médias e percentuais geralmente não devem ser somados; calculamos a partir de numeradores e denominadores aditivos.

Guardar apenas `ticket_medio` por loja impede recomposição correta do total. Duas lojas com tickets médios 50 e 100 não produzem média 75 se volumes diferirem. Guardamos receita e número de pedidos, então a camada semântica divide `SUM(revenue)` por `COUNT_DISTINCT(order_id)`. Essa regra preserva ponderação em qualquer filtro compatível.

Dimensão tempo é mais que extrair mês de uma data. Ela registra calendário, dia útil, semana, feriado e período fiscal, mantendo a data de evento escolhida. Um pedido possui várias datas com papéis diferentes; podemos ligar dimensões de papel, como data

do pedido e data da entrega. Nomear o papel evita usar mês de pedido em uma medida definida por entrega.

Dimensões conformadas permitem comparar canais. Produto do marketplace XP-55 e SKU interno 00055 precisam apontar para a mesma chave corporativa com regra e vigência. Canal também precisa de vocabulário: aplicativo próprio não deve aparecer como APP, Mobile e app loja. SOT resolve essas identidades antes de SPEC agregar.

Mudanças históricas pedem tipo de dimensão. Em uma dimensão lentamente mutável tipo 2, cada versão recebe chave substituta e intervalo de validade. A venda aponta para a versão vigente no evento. Isso preserva a pergunta histórica, mas aumenta cuidado no join temporal. Tipo 1 sobrescreve e serve quando somente o estado atual importa.

Fatos sem medida, como ocorrência de visita ou matrícula em campanha, ainda podem ser úteis para contagem. Tabelas ponte representam relações N:N controladas, como produto com múltiplas categorias, mas exigem regra de alocação para não somar receita inteira em cada categoria. Modelagem dimensional não elimina complexidade; torna a complexidade nomeável e testável.

A SPEC para receita diária pode selecionar as chaves de data de entrega, canal e produto, além da soma de valor bruto, desconto, devolução e contagem distinta de pedido. Para dashboard executivo, outra SPEC pode agregar por dia e canal. Ambas derivam do mesmo fato conformado e da mesma definição semântica, evitando que cada ferramenta reimplamente descontos e devoluções.

Tabela 5.5: Esboço do modelo dimensional da receita entregue.

Objeto	Grão	Campos centrais
Fato venda	item entregue	pedido, item, quantidade, bruto, desconto
Fato devolução	evento por item	item, data, estado, valor aprovado
Dim produto	versão de produto	SKU corporativo, categoria, vigência
Dim canal	canal conformado	loja, app, marketplace, responsável

Síntese da trilha

Então, aluno, veja só o que você aprendeu aqui: fatos preservam eventos e medidas; dimensões fornecem contexto conformado. Aditividade, papéis de data, histórico e pontes determinam se a exploração continuará correta ao mudar filtros.

5.7 SQL e pandas depois do desenho

Depois de definir receita, grão e cardinalidade, podemos implementar a SPEC. O SQL a seguir agrega itens entregues e devoluções aprovadas no grão de pedido antes do join. O objetivo didático não é decorar sintaxe, mas reconhecer onde o denominador e o nível de detalhe ficam explícitos.

```
WITH item_por_pedido AS (
  SELECT order_id,
         SUM(quantity * unit_price - discount_amount) AS item_net
  FROM sot_order_item
```

```

WHERE delivery_status = 'DELIVERED'
GROUP BY order_id
), devolucao_por_pedido AS (
    SELECT order_id,
           SUM(return_amount) AS returned
    FROM sot_return
    WHERE return_status = 'APPROVED'
    GROUP BY order_id
)
SELECT o.delivery_date,
       o.channel_key,
       SUM(i.item_net - COALESCE(r.returned, 0)) AS net_revenue,
       COUNT(DISTINCT o.order_id) AS delivered_orders
FROM sot_order o
JOIN item_por_pedido i ON i.order_id = o.order_id
LEFT JOIN devolucao_por_pedido r ON r.order_id = o.order_id
GROUP BY o.delivery_date, o.channel_key;

```

O 'JOIN' com itens é interno porque a SPEC de receita exige pedido com item entregue. O 'LEFT JOIN' com devolução preserva pedidos sem devolução, e 'COALESCE' transforma ausência de evento de devolução em zero somente porque a semântica foi estabelecida: não houve linha aprovada. Isso é diferente de imputar zero em um valor de devolução desconhecido.

Antes de publicar, executamos invariantes. A quantidade de pedidos da SPEC não pode superar pedidos entregues na SOT para o mesmo corte; receita bruta menos descontos menos devoluções precisa reconciliar com uma soma de referência; `channel_key` deve ter correspondência; datas futuras devem ser zero; cada lote deve informar atualização. Teste de dados é a forma executável do contrato.

Em pandas, o mesmo raciocínio aparece com 'groupby' antes de 'merge'. O parâmetro 'validate' verifica cardinalidade esperada e transforma uma suposição em falha explícita. Sem ele, a biblioteca aceita N:N e o resultado pode parecer normal.

```

item_order = (items.query("delivery_status == 'DELIVERED'")
               .assign(item_net=lambda d: d.quantity*d.unit_price-d.discount_amount)
               .groupby("order_id", as_index=False)["item_net"].sum())

return_order = (returns.query("return_status == 'APPROVED'")
                 .groupby("order_id", as_index=False)["return_amount"].sum())

base = orders.merge(
    item_order, on="order_id", validate="one_to_one")
base = base.merge(
    return_order, on="order_id", how="left",
    validate="one_to_one")
base["return_amount"] = base["return_amount"].fillna(0)
base["net_revenue"] = base["item_net"] - base["return_amount"]

```

A implementação precisa preservar chaves e colunas de auditoria mesmo que o painel não as mostre. Durante validação, agrupamos por lote, origem e estado; comparamos

linhas órfãs; amostramos pedidos; mantemos exemplos conhecidos como O101. Remover identificadores cedo dificulta explicar divergências e transforma o agregado em caixa-preta.

Código também deve lidar com moeda e precisão. Valores financeiros usam decimal com escala definida, não ponto flutuante binário sem controle. Taxas de câmbio precisam de fonte e data. Arredondar por item e arredondar depois da soma podem divergir; a política contábil deve estar no contrato e nos testes.

Consultas do dashboard não devem repetir toda a transformação. A SPEC materializa ou expõe visão governada; a camada semântica publica medidas reutilizáveis. O consumidor filtra e agrupa dentro dos limites previstos. Se cada workbook possui SQL customizado diferente, a organização cria várias fontes de verdade sem perceber.

Explicação guiada

Aluno, preste atenção aqui: ‘fillna(0)’ só é correto quando ausência significa semanticamente zero. No caso, falta de linha de devolução aprovada significa nenhuma devolução aprovada; falta de valor em uma devolução existente seria um problema diferente.

Síntese da trilha

Então, aluno, veja só o que você aprendeu aqui: SQL e pandas implementam um desenho anterior. Pré-agregação, cardinalidade validada, invariantes e precisão tornam a transformação reproduzível em vez de apenas executável.

5.8 Qualidade distribuída por camada

Qualidade de dados é adequação ao uso. Um campo pode ser completo e incorreto, válido e atrasado, único mas semanticamente duplicado. As dimensões mais úteis para este caso são completude, validade, unicidade, consistência, atualidade, integridade referencial e acurácia quando há referência externa. Cada uma exige métrica e limite ligado à decisão.

Na ingestão, verificamos chegada, esquema, volume, duplicidade de lote e legibilidade. Na SOR, preservamos contagens e hashes. Na SOT, testamos chaves, vocabulário, mapeamentos, integridade e regras temporais. Na SPEC, reconciliamos medidas, grão e filtros. No dashboard, monitoramos atualização, definição exibida e comportamento de acesso. Qualidade atravessa o fluxo.

Um teste de volume não pode usar limite fixo ingênuo. Vendas variam por dia da semana e campanha. Podemos comparar com faixa histórica por dia equivalente e manter tolerância documentada. Um alerta não prova erro; indica investigação. Ignorar toda variação para evitar falso positivo é tão perigoso quanto bloquear qualquer pico real.

Regras de validade derivam do domínio: quantidade positiva para venda, estado em conjunto permitido, entrega não anterior ao pedido, moeda conhecida, desconto dentro de política. Exceções legítimas precisam de representação, não de bypass manual. Um pedido de cortesia com valor zero pode ser válido se possuir tipo específico.

Reconciliação compara totais em pontos controlados. Número de itens da SOR por fonte deve explicar o da SOT depois de duplicatas e rejeições; receita da SOT deve explicar a SPEC depois de filtros e devoluções; o painel deve igualar consulta de referência. A diferença recebe ponte: entrada, removidos por regra, adicionados por correção e saída.

Observabilidade de dados acrescenta operação: frescor, falha de pipeline, distribuição

inesperada, linhagem impactada e incidentes. Um dashboard pode estar disponível e mostrar ontem como se fosse hoje. Exibir `updated_at` e estado de completude ajuda o usuário a avaliar confiança antes de decidir.

Responsabilidade precisa ser nominal. Dono de domínio define significado; steward cuida de qualidade e metadados; engenharia opera ingestão e transformação; analista valida adequação ao uso; consumidor contesta. Os títulos variam, mas a ausência de responsável transforma alerta em informação órfã.

Qualidade também inclui o que não sabemos. Se devoluções podem chegar por 30 dias, uma SPEC de agosto publicada em 1^o de setembro é preliminar. Rotular fechamento e janela de maturação é mais honesto que impor estabilidade artificial. A arquitetura deve permitir versões e explicar revisões.

Tabela 5.6: Controles distribuídos pelo caminho analítico.

Camada	Controle	Falha evitada
Ingestão	e lote, esquema,	perda ou repetição da captura
SOR	hash, volume	
SOT	chave, domínio,	entidades inconsistentes
	referência, tempo	
SPEC	grão, reconcilia-	KPI duplicado ou incompleto
	ção, filtros	
Consumo	frescor, definição	decisão com dado velho ou indevido
	e acesso	

Síntese da trilha

Então, aluno, veja só o que você aprendeu aqui: qualidade não é uma faxina entre fonte e dashboard. Cada camada possui riscos próprios, controles mensuráveis, limites e responsáveis; revisões legítimas também precisam ser visíveis.

5.9 Catálogo, linhagem e camada semântica

Catálogo é o sistema de descoberta e entendimento dos ativos de dados. Ele registra nome, descrição, dono, grão, esquema, classificação, qualidade, atualização e consumidores. Uma lista automática de colunas ajuda, mas não basta: `net_revenue` precisa de definição de negócio, unidade, filtros e exemplo.

Linhagem descreve como um dado nasce, é transformado e consumido. Linhagem técnica liga tabelas, colunas e jobs; linhagem de negócio liga medida, regra, responsável e decisão. Para a receita líquida, precisamos saber quais campos de item e devolução participam, qual versão da transformação gerou a SPEC e quais painéis serão afetados se a regra mudar.

Impacto percorre a linhagem para frente: se `return_status` muda, quais SPECs, medidas e painéis dependem dele? Investigação percorre para trás: por que o número de agosto mudou? Sem linhagem, equipes procuram manualmente arquivos e consultas; com linhagem incompleta, uma seta bonita pode sugerir segurança que não existe. Testamos links críticos com exemplos.

Camada semântica centraliza entidades, dimensões e medidas em vocabulário consumível. Ela pode publicar ‘Receita líquida entregue’, ‘Pedidos entregues’ e ‘Ticket médio

entregue’ com fórmulas, formatos, dimensões permitidas e política de acesso. O objetivo é evitar que Tableau, Streamlit, planilha e agente de IA implementem versões locais.

Uma medida semântica deve declarar aditividade. Receita soma por tempo, canal e produto se o fato estiver no grão correto; ticket médio é razão e precisa recalcular numerador e denominador após filtros. Percentual de devolução requer base explícita: itens, pedidos ou valor. A camada semântica protege essas operações e rejeita combinações sem sentido quando possível.

Fonte de verdade é um acordo governado, não apenas localização. A autoridade sobre entrega pode estar no sistema logístico, sobre pagamento no adquirente e sobre produto no cadastro mestre; SOT conforma essas autoridades. Para uma medida, a fonte de verdade inclui definição, dado, transformação, versão e responsável. Um arquivo copiado para a área de trabalho não vira verdade por estar mais perto do analista.

Nomes amigáveis não devem apagar rastreabilidade. O painel pode exibir “Receita líquida”, enquanto o catálogo liga a `net_revenue_delivered` e sua expressão. Sinônimos ajudam busca, mas termos conflitantes precisam de resolução. Se finanças e comercial usam “receita” de modos diferentes, publicamos medidas qualificadas em vez de escolher uma silenciosamente.

Para IA aplicada a AED, semântica e catálogo serão ainda mais importantes. Um modelo de linguagem pode mapear “quanto vendemos?” para qualquer coluna parecida. Com entidades, medidas aprovadas, descrições, exemplos e permissões, o sistema reduz ambiguidade e pode pedir esclarecimento. Linhagem permite anexar evidência à resposta e contestar o caminho executado.

Tabela 5.7: Ficha mínima da medida semântica.

Campo de catálogo	Conteúdo do caso
Nome	Receita líquida entregue
Grão/base	item entregue; agregável por data de entrega
Fórmula	bruto menos desconto menos devolução aprovada
Atualização	horária; devoluções revisam até 30 dias
Dono	domínio Financeiro, com validação Comercial

Explicação guiada

Aluno, preste atenção aqui: camada semântica não é trocar nomes técnicos por rótulos bonitos. Ela governa cálculo, grão, aditividade, filtros, acesso, versão e evidência para que consumidores diferentes obtenham a mesma resposta.

Síntese da trilha

Então, aluno, veja só o que você aprendeu aqui: catálogo explica o ativo, linhagem explica o caminho e semântica explica o significado operacional. Juntos, transformam uma tabela disponível em evidência encontrável e contestável.

5.10 Do modelo ao dashboard com acesso seguro

O dashboard consome SPEC e medidas semânticas, não deveria decidir novamente como deduzir devolução ou qual data define agosto. Sua responsabilidade é representar a pergunta, permitir cortes autorizados e mostrar contexto: período, unidade, atualização, filtros e limitações. Quanto mais regra escondida no workbook, mais difícil reconciliar outro consumidor.

Dimensões expostas devem respeitar o grão. Ao adicionar produto a uma SPEC agregada apenas por canal e dia, o campo não existe legitimamente; uma ferramenta não pode recuperar detalhe perdido. Podemos manter uma SPEC detalhada para exploração e outra agregada para desempenho, ambas com medidas alinhadas. O usuário precisa saber qual atende sua pergunta.

Filtros de segurança em nível de linha limitam registros conforme identidade, como gerente vendo apenas sua região. Mas segurança precisa existir no serviço ou camada de dados, não somente no visual. Exportação, consulta direta ou agente devem receber a mesma política. Ocultar uma aba não protege informação.

Dados pessoais seguem princípio de minimização. O painel de receita por canal não precisa de nome, e-mail ou telefone do cliente. Identificadores técnicos podem permanecer em camada restrita para auditoria, enquanto SPEC usa chave pseudonimizada ou nenhum identificador pessoal. Finalidade, retenção e acesso precisam ser documentados.

Cache e extract melhoram desempenho, porém introduzem versão e latência. Um extract do Tableau, cache do Streamlit ou resultado de API precisa informar quando foi atualizado e qual SPEC usou. Invalidar cache após correção é parte da operação. Dois painéis podem divergir simplesmente porque um conserva versão antiga.

Uma consulta conversacional não elimina o dashboard. A resposta textual “R\$ 8,4 milhões” precisa de período, medida, filtros, atualização e comparação. O visual ajuda a detectar cauda, quebra e composição; a interface conversacional ajuda a formular e aprofundar perguntas. Ambos devem usar a mesma camada semântica e a mesma política de acesso.

Critérios de publicação incluem reconciliação, desempenho, legibilidade, acessibilidade, segurança, dono e canal de contestação. Um selo verde permanente é insuficiente; qualidade pode degradar amanhã. O painel deve mostrar estado atual e permitir chegar à definição. Confiança é mantida por processo, não por aparência.

A entrega do capítulo pode ser representada pelo diagrama da Figura 5.1 acompanhado de uma ficha por etapa: grão, chave, medida, dono e controle. Para o caso, fonte traz chaves de origem; SOR, lote; SOT, pedido/item conformado; SPEC, receita por dia e canal; dashboard, definição e atualização. Essa ficha transforma arquitetura em argumento verificável.

Síntese da trilha

Então, aluno, veja só o que você aprendeu aqui: consumo seguro reutiliza medida e política, preserva contexto e expõe atualização. Dashboard e conversa com IA são interfaces sobre uma arquitetura, não novas fontes de verdade.

Tabela 5.8: Contrato mínimo da fonte ao consumo.

Etapa	Grão/chave	Dono	Controle decisivo
Fonte e SOR	chave original + lote	sistema e captura	completude e idempotência
SOT	entidade conformada	domínio de dados	unicidade e referência
SPEC	dia + canal	produto analítico	reconciliação da medida
Dashboard	marca por corte	dono do painel	frescor, acesso e definição

5.11 Fechamento: da divergência à contestação

Voltamos aos três números de receita. A divergência não era necessariamente defeito de cálculo; cada sistema observava evento, tempo e dedução distintos. Ao transformar a pergunta em contrato, conseguimos nomear receita bruta pedida, receita líquida entregue, caixa recebido e repasse. A clareza semântica veio antes da reconciliação técnica.

As fontes continuaram responsáveis por seus eventos. A ingestão preservou lotes, esquema e origem na SOR. A SOT conformou produto, canal, tempo e estados sem apagar valores originais. A SPEC materializou o grão e a regra necessários ao painel. Essa divisão permitiu reprocessar e explicar, em vez de corrigir manualmente o número final.

Grão, chaves e cardinalidade impediram que um join N:N duplicasse receita. A modelagem dimensional separou fatos e contextos, preservou numeradores e denominadores e tratou histórico. SQL e pandas vieram depois, com pré-agregação e validações. A execução passou a verificar uma decisão explícita.

Qualidade foi distribuída: lote e esquema na captura; identidade e domínio na SOT; reconciliação na SPEC; frescor e acesso no consumo. Catálogo registrou grão e dono, linhagem conectou origem e impacto, e camada semântica centralizou a medida. Fonte de verdade tornou-se acordo completo, não arquivo isolado.

O protocolo pode ser resumido assim: definir decisão e medida; inventariar autoridades; capturar com metadados; separar preservação, conformação e serviço; modelar grão e cardinalidade; testar qualidade e reconciliação; catalogar e ligar linhagem; publicar semântica e acesso; mostrar atualização e limite. Cada etapa responde a uma falha observável.

Na próxima semana, estudaremos o gráfico como operação perceptiva. A arquitetura correta não garante uma visualização correta: eixos, posição, comprimento, área e escala ainda podem induzir leitura equivocada. Contudo, sem a arquitetura de hoje, o gráfico mais claro apenas comunicaria com eficiência um número indefensável.

Síntese da trilha

Então, aluno, veja só o que você aprendeu aqui: arquitetura de dados para AED é a disciplina de manter significado e evidência entre pergunta e decisão. Quando o número for contestado, o caminho deve poder ser percorrido de volta até a fonte.